

Solidity Calculator Contracts

A walkthrough of ICALCULATOR.sol, calculator.sol, calculatorContract.sol, and newCalculator.sol — what each contract does, how they relate, and important issues to be aware of.

This document explains four connected Solidity files that together build a simple on-chain calculator system. The files demonstrate three core Solidity concepts:

- **Interfaces** — ICALCULATOR.sol defines a contract's public API without implementation.
- **Inheritance** — calculatorContract.sol inherits all logic from calculator.sol.
- **External contract calls** — newCalculator.sol talks to an already-deployed calculator through the ICALCULATOR interface, rather than inheriting from it.

1. ICALCULATOR.sol — The Interface

This file defines an **interface**, not a working contract. An interface only lists function signatures (name, parameters, visibility, and return type) — it contains no logic and no state variables. Think of it as a contract, a promise, or a menu: it says exactly what functions a contract must offer, but not how those functions behave internally.

```
interface ICALCULATOR {  
    function add(uint a) external;  
    function sub(uint a) external;  
    function multi(uint a) external;  
    function divide(uint a) external;  
    function result() external view returns (uint);  
    function clear() external;  
}
```

What it declares:

- **add / sub / multi / divide** — four arithmetic operations, each taking one uint argument.
- **result()** — a read-only (view) function that returns the current calculated value.
- **clear()** — resets the calculator's stored value.

Why it matters:

- Any contract that implements ICALCULATOR is guaranteed to expose these six functions with these exact signatures — this is what makes interfaces useful for interacting with contracts whose full source code you may not have, as long as you know their interface.
- Interfaces cannot contain implementation code, constructors, or state variables — every function must be marked external, and interfaces cannot inherit from regular contracts.

2. calculator.sol — The Implementation

This contract provides the actual logic for ICALCULATOR. It declares contract `calculator` is `ICALCULATOR`, meaning it promises to implement every function the interface defined — and it does.

```
contract calculator is ICALCULATOR {
    uint public temp;

    function add(uint a) external {
        temp += a;
    }
    function sub(uint a) external {
        temp -= a;
    }
    function multi(uint a) external {
        temp *= a;
    }
    function divide(uint a) external {
        temp /= a;
    }
    function result() external view returns (uint) {
        return temp;
    }
    function clear() external {
        temp = 0;
    }
}
```

What it does:

- **temp** — a public uint state variable that stores the running total. Because it's public, Solidity automatically generates a free getter function for it as well.
- **add/sub/multi/divide** — each mutates temp in place using compound assignment (`+=`, `-=`, `*=`, `/=`) based on the argument passed in.
- **result()** — simply returns temp; marked view since it doesn't change state.
- **clear()** — sets temp back to zero.

Points worth noting:

- Because temp is a uint (unsigned integer), it can never go negative. Calling `sub(a)` when a is greater than the current temp will cause the transaction to revert (Solidity 0.8+ has built-in overflow/underflow checks).
- `divide(0)` will also revert, since division by zero is not allowed.
- There's no constructor and no access control — anyone can call `add`, `sub`, `multi`, `divide`, or `clear` on a deployed calculator, and every call permanently changes shared state.

3. calculatorContract.sol — Inheriting the Logic

This contract does almost nothing on its own — and that's the point. It inherits directly from calculator, meaning it automatically receives every function and state variable that calculator has, without rewriting any code.

```
contract calculatorContract is calculator {  
  
}
```

What it does:

- By writing `is calculator`, this contract inherits `add`, `sub`, `multi`, `divide`, `result`, `clear`, and the public `temp` variable exactly as defined in `calculator.sol`.
- When deployed, `calculatorContract` behaves identically to `calculator` — it's effectively a second, independent instance of the same calculator logic, with its own separate `temp` value.

Why you'd do this:

- An empty inheriting contract like this is often used as a pattern for future extension — you could later add new functions or override existing ones (e.g. add access control, events, or new operations) without touching the original calculator contract.
- It also demonstrates that inheritance in Solidity works transitively: `calculatorContract` also satisfies the `ICALCULATOR` interface, because `calculator` already does.

4. newCalculator.sol — Calling an External Contract

This contract takes a completely different approach from calculatorContract. Instead of inheriting the calculator's code, it stores the **address** of an already-deployed calculator (or any contract implementing ICALCULATOR) and talks to it through the interface.

```
contract newCalculator {
    ICALCULATOR public calc;

    constructor(address calculatorAddress) {
        calc = ICALCULATOR(calculatorAddress);
    }

    function add(uint a) public {
        calc.add(a);
    }

    function result() public view returns (uint) {
        return calc.result();
    }

    function add1(uint a) public pure returns (uint) {
        return a + a;
    }
}
```

What it does:

- **calc** — a variable typed as ICALCULATOR. It doesn't hold contract code itself; it holds an address, wrapped so Solidity knows which functions are callable on it.
- **constructor(address calculatorAddress)** — runs once at deployment. It takes the address of an existing calculator contract and casts it to ICALCULATOR, storing it in calc.
- **add(uint a)** — does not do arithmetic itself. Instead it makes an external call, calc.add(a), which executes on the target calculator contract and modifies *that* contract's temp, not newCalculator's own storage.
- **result()** — likewise forwards the call to calc.result() and returns whatever the target calculator currently holds.
- **add1(uint a)** — unrelated to the calculator: a standalone pure function that simply returns a doubled (a + a), with no state access and no external call.

Why this pattern matters:

- This is the standard way to interact with another already-deployed contract on-chain: cast its address to the shared interface, then call functions on it as if it were a local object.
- newCalculator has no temp variable of its own for add/result — all arithmetic state lives in the external calculator contract it points to. newCalculator is essentially a thin 'proxy' or 'client' for that other contract.
- sub, multi, divide, and clear from ICALCULATOR were never re-exposed here — newCalculator only forwards add and result, so a user of newCalculator cannot subtract, multiply, divide, or clear the remote calculator through this contract (though they could still call the target calculator directly).

Summary: How the Four Files Relate

File	Role	Key Mechanism
ICALCULATOR.sol	Defines the shared API (no logic)	interface
calculator.sol	Implements the arithmetic logic and stores temp	implements the interface
calculatorContract.sol	A separate, independent calculator instance	inheritance (is calculator)
newCalculator.sol	Remotely controls an existing calculator	external call via interface casting

In short: ICALCULATOR is the contract (the promise of what functions exist). calculator fulfills that promise with real logic. calculatorContract reuses that logic through inheritance, producing its own independent instance. newCalculator does not inherit anything — it holds the address of an already-deployed calculator and drives it remotely through the ICALCULATOR interface, so any add/result calls on newCalculator actually change and read the state of that other, external contract.